The CRAZY 10'S

A Card Game for Teaching Dozenal (Base-12) Arithmetic

Team 25

Ruida Chen | Alvin Qian | Jiaming Li | Ammar Sharbat

THE FLAW OF BASE-10

Why Base-10 Fails

Human culture is built on **decimal** (Base-10), yet it is mathematically limited. 10 only has factors of **2 and 5**.

$$\frac{1}{3} = 0.333...$$

Historical disasters like the **Pentium FDIV bug (1994)** and **Ariane 5 crash (1996)** highlight the risks of floating-point errors in complex systems.

The Dozenal Solution

Base-12 has factors of 2, 3, 4, and 6, allowing for cleaner division and intuitive fractions.

- $\frac{1}{3} = 0.4$
- $\frac{1}{4} = 0.3$
- $\frac{1}{6} = 0.2$

PROJECT ROADMAP: MVP -> STRETCH

Project Roadmap

MVP Requirements

- Core Game Loop
- Dozenal System Support

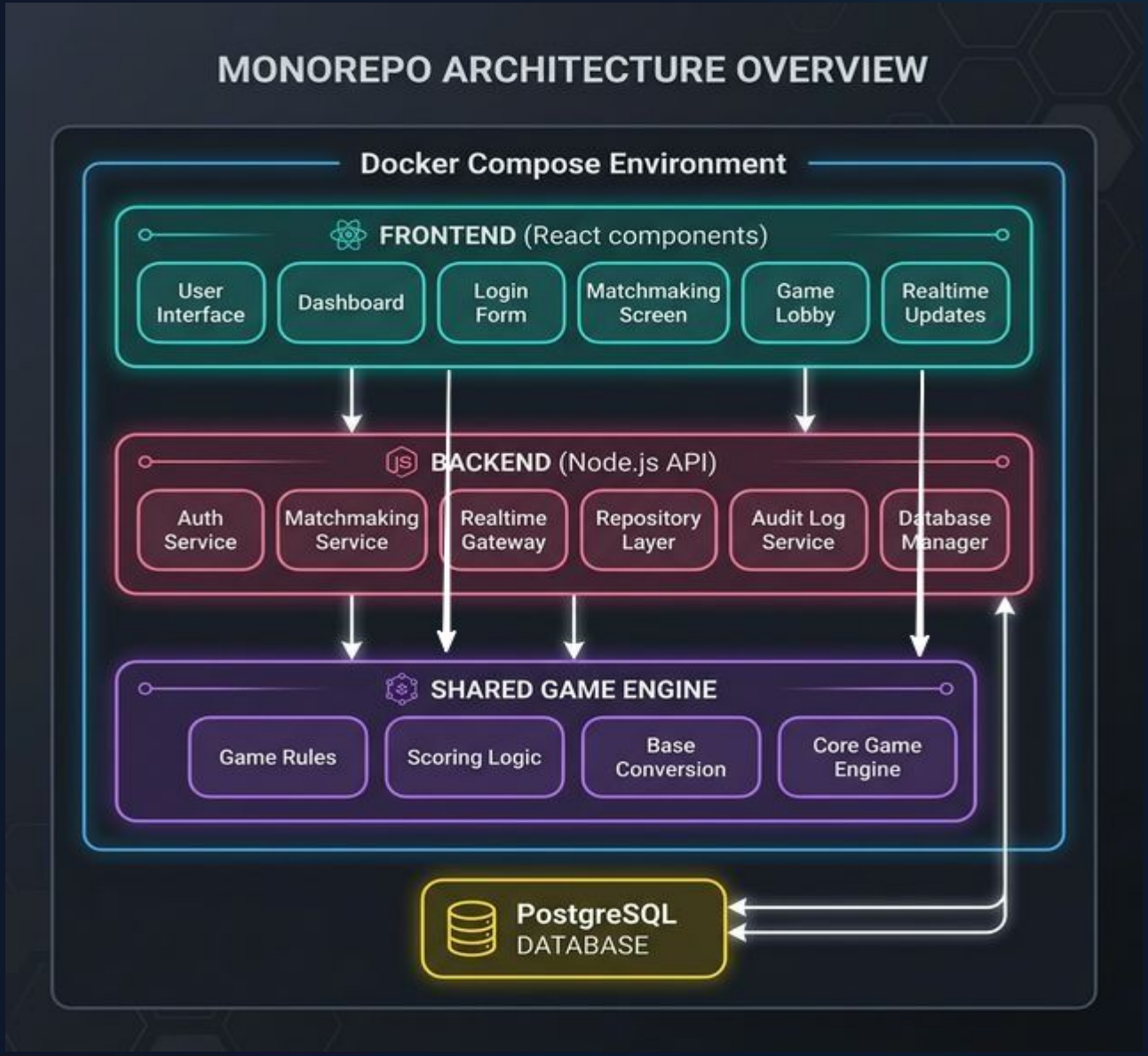
COMPLETED

Stretch Goals

- Online Multiplayer
- Arithmetic Challenge Engine
- User Profiles & Match History
- In-Game Chat
- Highlighted Playable Cards
- Custom Card Themes (*In Development*)

3-TIER SYSTEM ARCHITECTURE

- **Frontend:** Built with Vite, React, and TypeScript.
- **Backend:** Node.js + Express.js + Socket.io. Manages JWT Auth and Lobby persistence.
- **Shared Engine:** A TypeScript library acting as the "Single Source of Truth" for game rules.
- **Database:** Repository pattern allows hot-swapping between In-Memory (Dev) and PostgreSQL (Prod).



CORE GAMEPLAY

The Sum Rule

Players can play two cards that sum to the target value. (e.g., 5 + 7 = 10_{12}).

Dozenal Numerals

Full support for □ and □ with bi-directional conversion logic.

Math Races

Triggered by Face Cards. Real-time arithmetic challenges to gain score advantages.

LIVE SYSTEM DEMO

✅ 🖥️ **Frontend (Client)**

- TypeScript
- High-performance, dynamic UI rendering.

✅ ⚙️ **Backend (Server)**

- Node.js + Socket.io
- Manages real-time state, JWT Auth, and lobby persistence.

✅ 🔗 **Shared Engine (The "Single Source of Truth")**

- Standalone TypeScript library.
- Guarantees 100% rule sync between client and server.

DEPLOYMENT & DEVOPS

GitHub Actions

Infrastructure & Runtime Stack

Architecture: 3-container Docker Compose setup (Frontend, Backend, PostgreSQL)

Hosting: Microservices on AWS EC2 via Nginx Reverse Proxy

Security: JWT session management and environment-variable config

GitHub Actions CI/CD

Continuous Integration

Pipeline triggered on push/PR. Automated builds for shared packages, backend, and frontend with dependency caching.

Automated Test Gates

Shared test suite runs with coverage reporting. Artifacts are uploaded upon successful validation of multiplayer state transitions.

Docs as Code

CI compiles LaTeX source into updated PDFs on every documentation change, ensuring repository-backed manuals stay current.

Next Phase: Production CD

Automating the transition from CI success to production deployment on EC2. Current state handles automation up to build/test artifacts.

REPO REFERENCES: Node CI Workflow • LaTeX Docs Workflow • Container Orchestration • Nginx Proxy Config • Env Template

REFLECTIONS & LESSONS

Successes

- The **Shared Monorepo Package** was critical. It prevented rule desyncs between frontend and backend.

- The **Repository Pattern** allowed us to develop offline and deploy to cloud with zero code changes.

Growth Areas

- Implementing **TypeScript Strict Mode** earlier would have caught edge cases in Socket payloads.

- Adding **E2E Testing** (Playwright) would further stabilize the complex multiplayer state transitions.

THE FUTURE OF CRAZY 10'S

	Bases 1-10		Bases 11-20				
1	unary	11	undecimal	21	unvigesimal	40	quadragesimal
2	binary	12	duodecimal	22	duovigesimal	50	quinquagesimal
3	ternary	13	tridecimal	23	trivigesimal	60	sexagesimal
4	quaternary	14	tetradecimal	24	tetravigesimal	70	septuagesimal
5	quinary	15	pentadecimal	25	pentavigesimal	80	octogesimal
6	senary	16	hexadecimal		⋯	90	nonogesimal
7	septenary	17	heptadecimal	30	trigesimal	100	centesimal
8	octal	18	octodecimal	32	duotrigesimal	120	centovigesimal
9	nonary	19	enneadecimal	36	hexatrigesimal	144	centotetraquadragesimal
10	decimal	20	vigesimal		⋯	360	trecentosexagesimal

Expansion

Implementing local AI bots using MCTS.

New Bases

Support for Hexadecimal (Base-16) and Octal (Base-8) training.

Mobile Port

React Native application for iOS and Android platforms.

New Mode

Design and implement new game mode for more fun! (Roguelike, infinite mode…)

THANK YOU!

Team 25

Ruida Chen | Alvin Qian | Jiaming Li | Ammar Sharbat